1

# Resource Relation Patterns

Managing associations between resources



Includes content adapted from *API Design Pattern*, 2021, Manning Publications && *RESTful Java Web Services, 3rd Ed.* , 2017, Packt Publishing



## Resource Relation Patterns

1. **Singleton Sub-Resources**
2. Cross References
3. Association Resources
4. Add and Remove Custom Methods
5. Polymorphism

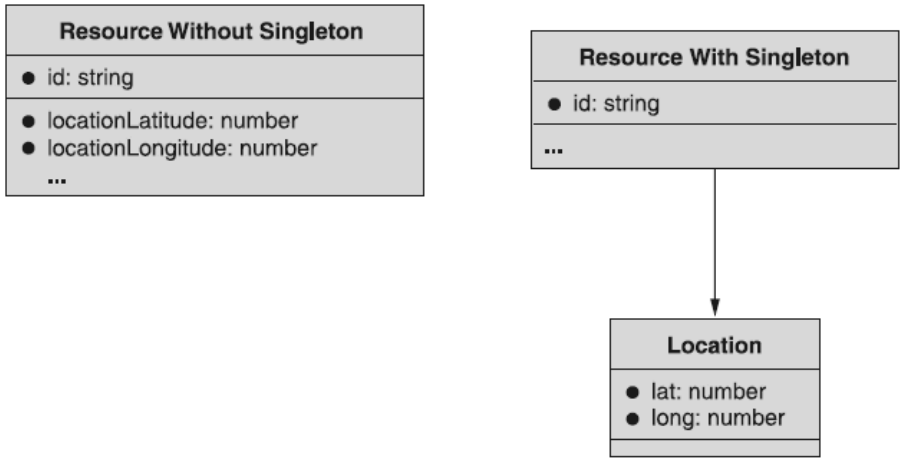


## Singleton Sub-Resources

- Sometimes we can identify some data that is obviously a property of a resource, but for some reason it is not practical to implement it as such. Reasons may include:
  - ***Size and complexity*** – The sub-resource is much larger than the rest of the resource.
  - ***Security*** – The sub-resource has different security requirements (*e.g. public employee data vs employee payment data*).
  - ***Volatility*** – The sub-resource has different access patterns (*e.g. it is very frequently updated – Driver Personal Data vs Driver Location*)

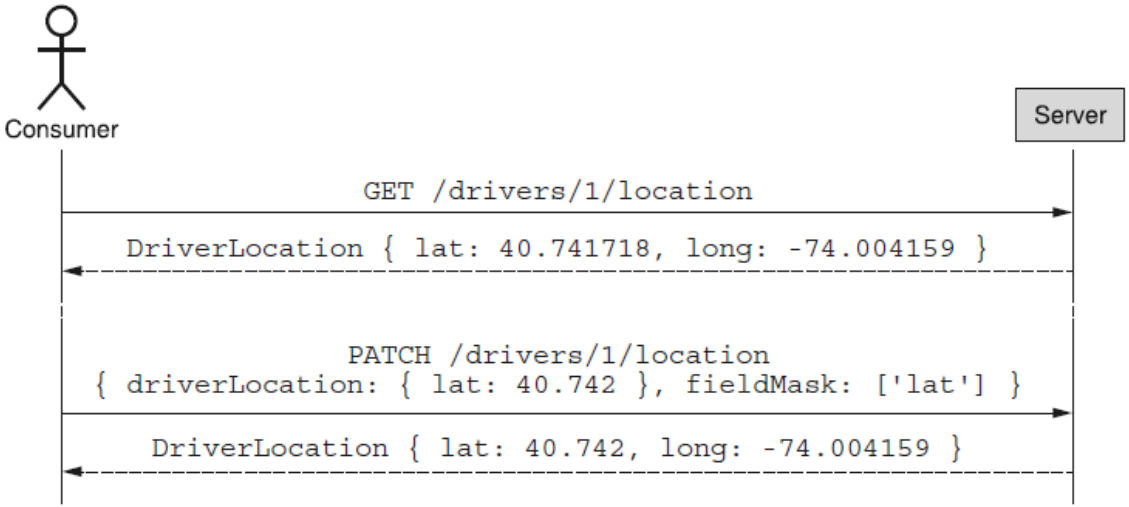
# Singleton Sub-Resources

- A solution for this is to create a sub-resource that behaves as an hybrid of a full resource and an attribute:



# Singleton Sub-Resources

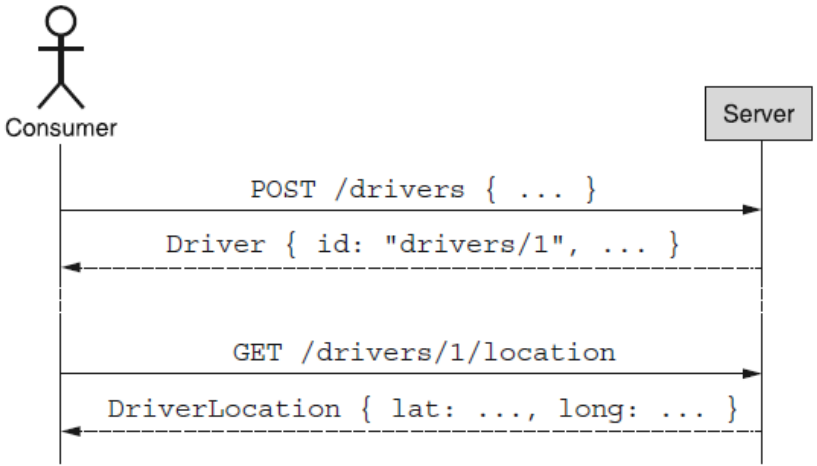
- **GET** and **UPDATE**: The singleton sub-resource behaves as a standard resource.
- It is uniquely addressable and can be modified in the traditional way.




**Note:** GETing the parent resource will generally not return sub-resource data (by design).

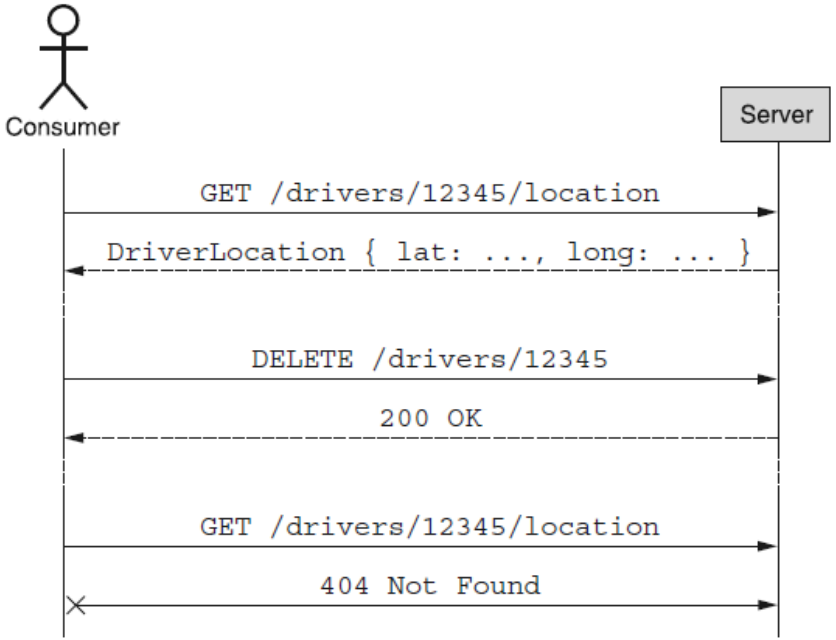
# Singleton Sub-Resources

- **Create**: The singleton sub-resource behaves as a standard property.
  - It is not explicitly created.
  - It is created when the parent resource is created.
- The sub-resource is created using reasonable defaults. It should <u>not</u> possible to explicitly initialize it to specific values.
  - *The point of the pattern is to isolate the parent and sub-resource.*



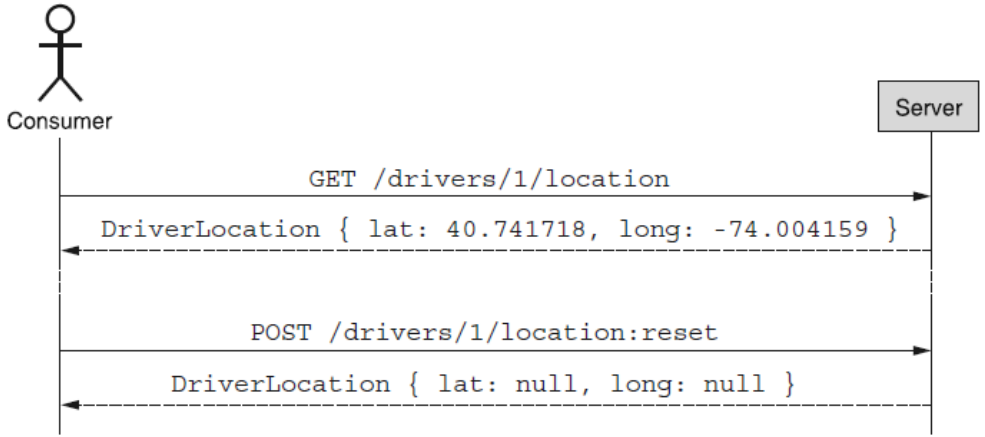
# Singleton Sub-Resources

- **Delete**: The singleton sub-resource behaves as a standard property.
  - It is deleted when the parent resource is deleted.



# Singleton Sub-Resources

- **Reseting**: It might be useful to provide a means to reset the singleton sub-resource to the default initialization values.
  - This can be achieved by implementing a custom `reset` function.



## Resource Relation Patterns

1. Singleton Sub-Resources
2. **Cross References**
3. Association Resources
4. Add and Remove Custom Methods
5. Polymorphism



# Cross-References

- Resources will often need to cross-reference other resources. There are several approaches to do this and issues to consider.
  - How to reference those resources?
  - What kind of operations can I perform on cross-referenced resources?
  - How to deal with consistency issues (e.g. the deletion of cross referenced resource)?



# Cross-References

- As expected, cross referencing will rely on a property that stores the unique identifier of the referenced resource.

```
interface Book {
  id: string;
  authorId: string;
  title: string;
  // ...
}

interface Author {
  id: string;
  name: string;
}
```

We store a reference to the Author resource using a string field holding a unique identifier of an author.

We'll leave out extra fields for brevity.

The unique identifier of the Author resource. This is the value that appears in the authorId field.



# Cross-References

- In some cases, the type of resource might be dynamic. In this case, an additional type property is required.

```
interface ChangeLogEntry {
  id: string;
  targetId: string;
  targetType: string;
  // ...
}
```

The unique identifier of the target resource (e.g., the author ID)

The type of the target resource (e.g., api.mycompany.com/Author)

Here we'd store more details about this particular change to the target resource.



# Cross-References

- **Data Integrity:**
  - The flexibility of untyped string references allows data inconsistencies to manifest.
  - How to deal with the following scenario?

```
author = CreateAuthor({ name: "Michelle Obama" });
book = CreateBook({
  title: "Becoming",
  authorId: author.id
});
DeleteAuthor({ id: author.id });
```

Here we create an author and then a book referring to that author.

After that, we delete the Author resource from our API.



# Cross-References

```
author = CreateAuthor({ name: "Michelle Obama" });
book = CreateBook({
  title: "Becoming",
  authorId: author.id
});
DeleteAuthor({ id: author.id });
```

Here we create an author and then a book referring to that author.

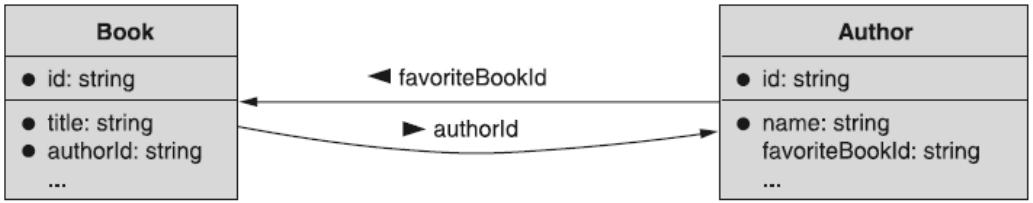
After that, we delete the Author resource from our API.

- There are a few alternative options...
  - **Option 1.** Prohibit the deletion and throw an error.
  - **Option 2.** Allow deletion and set the `authorId` property to a zero/null value.
  - **Option 3.** Allow deletion and deal with bad pointer at runtime.



# Cross-References

- **Option 1.** Prohibit the deletion and throw an error.
  - Highly inconvenient for user. To delete an author, it may be necessary to delete hundreds or thousands of books first.
  - Also, this approach makes it impossible to delete resources with circular references.





## Cross-References

- **Option 2.** Allow deletion and set the `authorId` property to a zero/null value.
  - We may need to update a potentially large number of records before allowing deletion. This may take a while, and if not done atomically, may end up leaving dangling pointers anyway.
  - Violates assumptions of standard DELETE method: atomicity and no side effects.
  - This leaves us with the third option...



## Cross-References

- **Option 3.** Allow deletion and deal with bad pointer at runtime.
  - Ask API consumers to expect that reference fields might be invalid or point to resources that may have been deleted.
  - Can be inconvenient, but allows consistent behaviour: all references must be checked.
  - Does not violate standard method constraints (atomicity & no side effects)



# Cross-References

- **Value vs. References:**
  - Rather than relying on returning references, it is also possible to embed the a cached value of the cross-referenced resource.

```
{
id: string,
title: string,
author: {
        id: string,
        name: string
        }
}
```

VS

```
{
id: string,
title: string,
authorId: string
}
```



# Cross-References

- **Value vs. References:**
  - An obvious advantage of using the value instead of reference is reducing the number of required API calls.
  - Using the “value” approach is therefore very convenient.

```
book = GetBook({
  "id: "books/1234"
});
authorName = GetAuthor({ id: book.authorId }).name;
```

Here we retrieve a book by a (random) unique identifier.

To see the name of the author, we need a separate API call.



# Cross-References

- **Value vs. References:**
  - **How to populate the value data on the server side?**
    - Retrieve author information at same time as request for book.
      - *Puts more load on database.*
      - *Ensures information is fresh when retrieved*
    - Store cached author information on book resource.
      - *Avoids overloading database,*
      - *But introduces consistency and freshness issues.*



# Cross-References

- **Disadvantages of the value-based approach:**
  - **The size of the response will grow every time the author resource grows.**
    - *This does not scale well, especially if the same strategy is adopted consistently for other aspects (e.g. publisher information).*
  - **This introduces some confusing, non-standard behaviour.**
    - *Can the user update information about an author when making changes to a book? Do we really want this to be a possibility?*
  - **<u>These potential issues makes the use of references the safer option.</u>**
    - *If necessary, dedicated libraries such as `GraphQL` can be used to allow a single request to retrieve a combination of information dispersed over multiple resources.*

## Resource Relation Patterns

1. Singleton Sub-Resources
2. Cross References
3. **Association Resources**
4. Add and Remove Custom Methods
5. Polymorphism

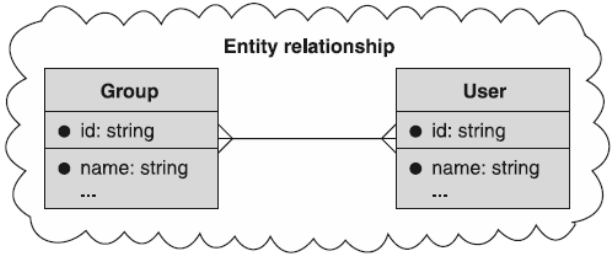


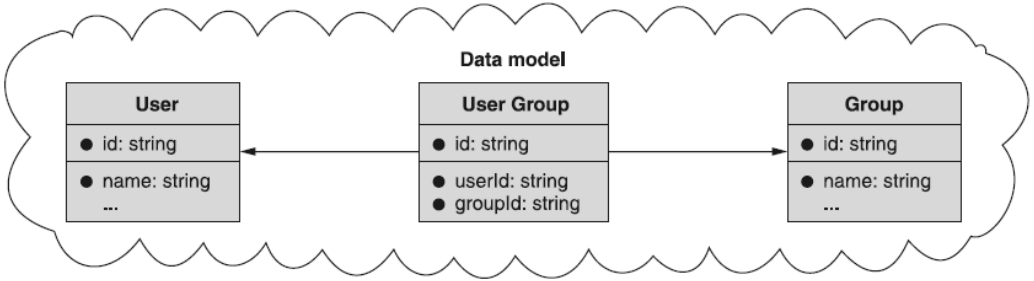
## Association Resources

- How to model many-to-many resource associations?
- How to associate additional information to each individual relationship between two resources?
- One approach is to use dedicated *Association Resources*.

# Association Resources

- Many-to-many associations are implemented with a *join table* on database systems.
- *Association resources* expose rows of this table to be directly manipulated by the API.





# Association Resources

- Association resources are created, deleted and manipulated just like any other resource:
  - **Creating a new resource:** User joining a group
  - **Deleting a resource:** User leaving a group
  - **Listing (with filter):** Identifying all users from a specific group
- Additional meta-data can be stored in the association resource for each individual relationship:
  - E.g. date the user joined that specific group.



# Association Resources

- Association resources are created, deleted and manipulated just like any other resource:

| Standard method | Behavior | Required? |
|---|---|---|
| Create | Joins the two resources | Yes |
| Get | Shows metadata about the relationship | Only if there's metadata |
| List | Lists the associations between the resources | Yes |
| Update | Updates metadata about the relationship | Only if there's metadata |
| Delete | Removes the association between the resources | Yes |



# Association Resources

- **Uniqueness constraint:**
  - Unlike normal resources, association resources cannot be duplicated.

```
let membershipData = { userId: "Jimmy", groupId: "2" };
let membership1 = CreateMembership(membershipData);
let membership2 = CreateMembership(membershipData);
```

First we add Jimmy to group 2. This should succeed.

Jimmy is already a member of group 2, so this request should fail with a 409 Conflict HTTP error.



## Association Resources

- **Read-only fields**
  - An association resource can be updated, but only the meta-data can change.
    - *An update should not make a different user be a member of the group.*
    - *An update should not make the user a member of another group.*
  - The corresponding fields should be considered read-only, and any update to their content should be ignored.



# Association Resources

- **Association alias methods:**
  - Sometimes, there is a sense that one of the resources involved in the association is the natural owner of the association.
    - I.e. A group has a list of member users.
  - It is natural to want to enquire about the status of the association:
    - What users belong to this group?
    - What groups does this user belong to?



# Association Resources

- **Association alias methods:**
  - Even though a filtered LIST is able to provide these answers, it is more natural for the user to think of these as questions directed to the resources (and not their association)
  - Convenient aliases can then optionally be created to allow users to make these questions using a single, unfiltered request.

```
GET /groups/1/members
GET /users/1/groups
```



# Association Resources

- **Referential Integrity:**
  - When resources are deleted, there are several options

| Option | Behavior |
|---|---|
| Cascade | Delete all other resources that point to the deleted resource. |
| Restrict | Prevent deleting any resources that are still pointed to by others. |
| Set null | Any pointers to the resource being deleted are set to null. |
| Do nothing | Deleting resources does nothing to the pointers and leaves them as invalid. |



# Association Resources

- **Referential Integrity:**
  - These options tend to scale poorly …

| Option | Behavior |
|---|---|
| Cascade | Delete all other resources that point to the deleted resource. |
| Restrict | Prevent deleting any resources that are still pointed to by others. |
| Set null | Any pointers to the resource being deleted are set to null. |
| Do nothing | Deleting resources does nothing to the pointers and leaves them as invalid. |



# Association Resources

- **Referential Integrity:**
  - … So one of these is preferable

| Option | Behavior |
|---|---|
| Cascade | Delete all other resources that point to the deleted resource. |
| Restrict | Prevent deleting any resources that are still pointed to by others. |
| Set null | Any pointers to the resource being deleted are set to null. |
| Do nothing | Deleting resources does nothing to the pointers and leaves them as invalid. |

  - **Restrict:** return precondition failed error (HTTP 412)
  - **Do nothing:** just delete the resource anyway.



## Resource Relation Patterns

1. Singleton Sub-Resources
2. Cross References
3. Association Resources
4. **Add and Remove Custom Methods**
5. Polymorphism



## Add and Remove Custom Methods

- An alternative pattern to implicitly handle many-to-many relationships.
  - It does not require the introduction of a new resource to manage the association.
- This approach is simpler than using association resources, with the trade off of having some limitations.



## Add and Remove Custom Methods

- In its simplest form, the association is managed by using custom `add` and `remove` methods instead.
  - These methods are shortcuts for creating and deleting associations.
  - They hide all the details of the association (except if it exists or does not exist).
- For example, if users can be members of multiple groups, these methods would allow users to join (`add`) and leave (`remove`) groups.



# Add and Remove Custom Methods

- **Limitations of this approach:**
  - We only store the association itself, not meta-data.
  - One of the resources must be identified as the ***managing resource***.
    - Do we add users to groups or groups to users?
    - Sometimes, the choice is obvious. Other times, not so much...



# Add and Remove Custom Methods

- **Implementation:**
  - These methods are applied to the managing resource and receive a reference to the other resource.

| Action | Method | HTTP equivalent |
|---|---|---|
| Add user 1 to group 1. | `AddGroupUser({ parent: "groups/1", userId: "users/1" });` | `POST /groups/1/users:add { userId: "users/1" }` |
| Remove user 1 from group 1. | `RemoveGroupUser({ parent: "groups/1", userId: "users/1" });` | `POST /groups/1/users:remove { userId: "users/1" }` |



# Add and Remove Custom Methods

- **Listing associated resources:**
  - We can resort to custom methods similar to the alias we defined for resource associations.

| Action | Method | HTTP equivalent |
|---|---|---|
| What users are in group 1? | `ListGroupUsers({ parent: "groups/1" })` | `GET /groups/1/users` |
| What groups is user 1 a member of? | `ListUserGroups({ parent: "users/1" });` | `GET /users/1/groups` |



# Add and Remove Custom Methods

- **Data integrity**
  - **What to do when adding the same resource twice or removing inexistent resource?**
    - Behaviour should be similar to deleting inexistent resource or creating a duplicate, e.g:
      - *Duplicate resource - HTTP 409 Conflict error*
      - *Inexistent resource - HTTP 412 Precondition failed*
  - **If the user just wants to ensure that “Jimmy” is a part of a group, then a successful result or conflict result are both valid results.**



## Resource Relation Patterns

1. Singleton Sub-Resources
2. Cross References
3. Association Resources
4. Add and Remove Custom Methods
5. **Polymorphism**



# Polymorphism

- Polymorphism is an important mechanism in software development.
  - *Use a common interface to access different types of objects with different behavior*
- How can we translate polymorphic behaviour into a resource-oriented API design?



## Polymorphism

- **Polymorphic Resources** represent a common generic interface (e.g. `Shape`) with an explicit field indicating the exact type of object (`Square`, `Triangle`).
  - *One advantage is that it is not necessary to duplicate standard methods for all type of sub-objects.*
- This strategy does not presume any specific implementation strategy for the resource, specifically about the data storage.
  - *A square has a side, a triangle has a base and height…*
  - *A common field may be reinterpreted depending on data type.*
  - *Alternatively, the structure may be interpreted dynamically.*



# Polymorphism

- **When to use polymorphic resources?**
  - Our concern when working in a local programming language is about generality and supporting future expansion and code reuse
  - In a web API, our concern should be more about how we intend the various standard methods to work and whether those standard methods operate identically on all of these resources or have some behavioural differences for the different types.



# Polymorphism

- **When to use polymorphic resources?**
- For example, consider if it makes sense to list all the different types of resource together:
  - In a chatroom app, almost certainly we want to be able to list all the `Messages`, regardless of their type (`VideoMessage`, `TextMessage`, etc).
  - To do otherwise would be cumbersome and would require multiple API calls, and then interleaving the results by creation time.



# Polymorphism

- **The type field:**
  - Is simply a string identifying the resource type, which is set to onne of the possible alternatives.
  - It should be unmutable: objects should not magically change their type once defined.

```
interface Message {
  id: string;
  sender: string;
  type: 'text' | 'photo' | 'audio' | 'video';
  // ...
}
```

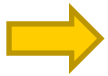
The type field is just a simple string storing the allowed subtypes.



# Polymorphism

- **Data validation**
  - Data validation should be enforced depending on the type of object.
  - Extraneous information should be ignored, as long as the required information is present:

```
create-Shape(
{ type: 'square',
  dimension: {
        radius: 10,
        length: 10
         }
}
```



Ok, ignore radius information



# Polymorphism

- **Polymorphic behaviour**
  - This setup easily allows us to create methods that exhibit different behaviour for different objects:

```
/shapes/id=1:countSides
```

Returns `{sides:3}`, or `{sides:4}`, etc, according to object type.